ENTREGA 2 — REPOSITORI DEVOPS I EVIDÈNCIES



Arman Ali
2n Daw

LINK Repo

PART C. APLICACIÓ DELS SKILLS AL PROJECTE WEB

A partir d'aquest punt hauràs d'utilitzar els coneixements adquirits als tutorials per transformar el projecte DevTasks.
La implementació s'ha de realitzar **pas a pas i en l'ordre indicat**.

PAS 1. Crear el repositori del projecte

Crea un repositori nou a GitHub per al projecte.
El repositori haurà de tenir:

- un nom identificatiu;
- un `README.md`;
- una branca principal `main`;
- el projecte web de partida.
  ATENCIÓ!!! L'app de partida té un bug de codi. El podeu comprobar desplegant l'app en local amb LiveServer i creant una tasca: veureu que la tasca no es crea correctamente. Trobeu el bug en el codi i corregiu-lo.
  Pista: compareu la declaració de la funció createTask amb la seva crida…

Evidència
Hauràs de poder demostrar:

- existència del repositori;
- estructura de fitxers;
- historial de commits.

PAS 2. Preparar el projecte JavaScript

Abans de començar amb GitHub Actions, **prepara el projecte per poder executar tests localment**.

1. **Instal·la NodeJs** al teu ordinador (https://nodejs.org/en) El necessitarem per fer servir el gestor de paquets `npm`, que ve inclòs en la instal·lació de NodeJs.
2. Després d'instal·lar NodeJs instal·la les dependències del projecte descrites al fitxer Package.json. Fes-ho amb la comanda (a l'arrel del projecte):

```
npm install
```

3. A la carpeta ***test/*** hi ha un fitxer ***.test.js*** que té definida una suite de testos d'algunes funcions clau de l'aplicació. Comprova que els tests es poden executar amb:

```
npm test
```

Identifica quines funcions es testegen i quin és el resultat del test.

PAS 3. Refactoritzar el JavaScript

Abans d'automatitzar els tests hauràs de **separar la lògica de negoci de la manipulació del DOM.**

La versió inicial conté tota la funcionalitat a:

`js/app.js`

Hauràs de refactoritzar-la.

L'objectiu és arribar, com a mínim, a:

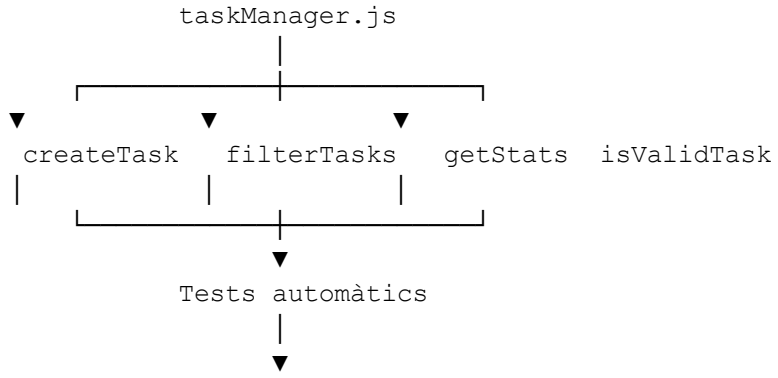
```
js/
├── app.js
└── taskManager.js
```

`taskManager.js`

Haurà de contenir la lògica que pugui ser executada i testada independentment del navegador. Cal moure a taskManager.js les funcions purament de lògica de negoci, i que són independents del DOM. Això farà que:

- Es puguin **testejar aquestes funcions sense error**.
- Es **separin les responsabilitats** de cada mòdul: a *app.js* quedarà tot el que té a veure amb la interfície gràfica i a *taskManager.js* tot el que té veure amb càlculs i comprovacions independents de la interfície i – per tant- reutilitzables en altres contextos.

L'objectiu és aconseguir:



PAS 4. Crear i ampliar els tests

Crea tests per comprovar el comportament de les funcions de ***`taskManager.js`***. Tingues en compte que, després de la refactorització, el fitxer de tests haurà d'incloure, a més de les funcions necessàries pel test:

```
import { describe, it, expect } from 'vitest';
```

les importacions de les funcions de *taskManager.js* a testejar:

```
import { isValidTask, createTask, filterTasks, getTaskStats } from '../js/taskManager.js';
```

Afegeix un test nou per a cada funció. **A classe explicarem l'arquitectura que tenen les funcions de test**: describe (it, expect, assert).

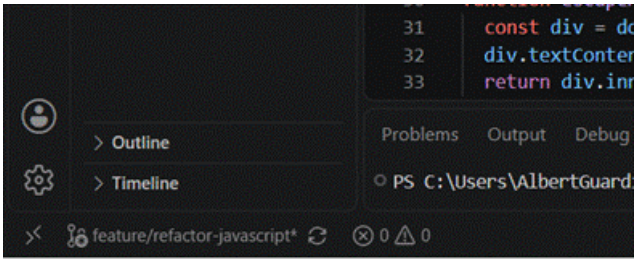
Executa els tests localment:

```
npm test
```

Abans de continuar, tots els tests han de passar.
En aquest moment encara estàs executant els tests localment. Posteriorment aprendràs a fer que GitHub els executi automàticament.

PAS 5. Crear una branca de funcionalitat

A partir d'aquest moment no hauràs de treballar directament sobre `main`.
Crea una branca `feature/refactor-javascript` (ho pots fer des del teu compte de GitHub.com), per exemple:
A VSCODE, assegura't que canvies de branca.



Puja a aquesta branca l'estat actual del projecte (bug corregit i tests afegits).

PAS 6. Crear una Pull Request

Quan hagis acabat la modificació:

1. Puja la branca a GitHub.
2. Crea una Pull Request cap a `main`.
3. Descriu els canvis realitzats.
4. Revisa els fitxers modificats.

La Pull Request serà el punt d'entrada del nostre procés d'Integració Contínua.

PAS 7 i 8. Crear el primer workflow de GitHub Actions

Ara hauràs d'automatitzar una tasca que fins ara feies manualment.
Crea:

```
.github/
└── workflows/
    └── ci.yml
```

El workflow haurà de:

1. executar-se quan es faci una Pull Request cap a `main`;
2. obtenir el codi del repositori;
3. preparar Node.js;
4. instal·lar les dependències;
5. executar els tests.

El fitxer YAML hauria de ser quelcom semblant a això:

Assegura't de pujar els canvis al repo de GitHub. El resultat haurà de ser visible a GitHub Actions.

PAS 9. Provocar un error deliberadament

Per demostrar que entens el funcionament del pipeline, hauràs de provocar un error controlat. Per exemple:

1. Modifica temporalment una funció.
2. Crea una Pull Request.
3. Comprova que GitHub Actions detecta el problema.
4. Observa que els tests fallen.
5. Corregeix el problema.
6. Torna a executar el pipeline.
7. Comprova que els tests tornen a passar.

Evidència obligatòria

Hauràs de capturar:

```
PR
 ↓
GitHub Actions
 ↓
TEST FAILED
```

i posteriorment, un cop arreglis el bug i tornis a pujar el codi:

```
PR
 ↓
GitHub Actions
 ↓
TEST PASSED
```

Aquesta serà una de les evidències més importants de la pràctica.

PAS 10. Protegir la branca `main`

Configura el repositori perquè `main` no pugui rebre directament els canvis sense passar pel procés establert.
Com a mínim hauràs d'aconseguir:

- Pull Request obligatòria;
- workflow de CI obligatori;
- no permetre fer merge si el CI falla.

L'objectiu és implementar:

```
CI PASS
   ↓
MERGE PERMÈS

CI FAIL
   ↓
MERGE BLOQUEJAT
```

Pas a pas per a la configuració a GitHub

1. Al teu repositori de GitHub, ves a la pestanya superior **Settings**.
2. Al menú lateral esquerre, selecciona **Branches** (o **Rulesets** segons la interfície de la teva organització).
3. A la secció **Branch protection rules**, fes clic al botó **Add branch ruleset** o **Add rule**.
4. A **Branch name pattern**, escriu exactament:
   Plaintext
   main
5. Marca les següents caselles de protecció obligatòries:
   - ✅ **Require a pull request before merging** (Exigeix una PR abans de poder incorporar canvis).
   - ✅ **Require status checks to pass before merging** (Exigeix que els comprovants de la CI hagin passat).
6. En la barra de cerca de status checks que s'obrirà a sota, cerca i selecciona el nom del job del teu workflow:
   Plaintext
   Install Dependencies and Run Tests
7. *(Opcional)* Marca la casella **Do not allow bypassing the above settings** per aplicar la regla als administradors del repositori.
8. Fes clic a **Create** o **Save changes** al final de la pàgina.

PAS 11. Configurar GitHub Pages

Publica l'aplicació mitjançant GitHub Pages.
L'aplicació haurà de poder consultar-se mitjançant una URL pública.
Comprova que:

- la pàgina inicial funciona;
- CSS es carrega;
- JavaScript funciona;
- les funcionalitats de l'aplicació són operatives.

PAS 12. Automatitzar el desplegament

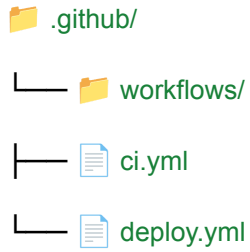
No volem haver de publicar manualment l'aplicació si la PR es resol amb un MERGE.

Pas 1. Activar GitHub Pages al repositori

1. Obre el repositori a GitHub i ves a la pestanya **Settings** (Configuració).
2. Al menú lateral esquerre, entra a la secció **Pages** (dins de *Code and automation*).
3. A la secció **Build and deployment > Source**, canvia l'opció per defecte (*Deploy from a branch*) i selecciona **GitHub Actions**.

Pas 2. Crear el workflow .github/workflows/deploy.yml

Al teu entorn local (o des d'una nova branca de treball), crea el fitxer de workflow per al desplegament:



El contingut de deploy.yml:

```yaml
name: Deploy Web App to GitHub Pages

# Trigger: S'executa només quan es fa un PUSH o MERGE directament a la branca main
on:
  push:
    branches:
      - main

# Permissos necessaris per poder escriure i desplegar a GitHub Pages
permissions:
  contents: read
```

```yaml
  pages: write
  id-token: write

# Garanteix que només hi hagi un desplegament actiu a la vegada
concurrency:
  group: 'pages'
  cancel-in-progress: true

jobs:
  deploy:
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    runs-on: ubuntu-latest

    steps:
      # Step 1: Descarrega el codi de la branca main
      - name: Checkout repository
        uses: actions/checkout@v4

      # Step 2: Configura l'entorn de Pages
      - name: Setup Pages
        uses: actions/configure-pages@v4

      # Step 3: Empaqueta i puja els fitxers estatics (.html, .css, .js) com a artefacte
      - name: Upload artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: '.' # Publica tot el directori arrel

      # Step 4: Desplega l'artefacte a l'entorn oficial de GitHub Pages
      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

Aquest workflow haurà de permetre desplegar automàticament la versió de `main` a GitHub Pages.

Pas 3. Executar i verificar el desplegament

1. Fes commit i push d'aquest fitxer **a la branca main** (o fes-ho mitjançant una PR i accepta el *Merge* a main).
2. Ves a la pestanya **Actions** de GitHub i observa el workflow **Deploy Web App to GitHub Pages**.
3. Un cop finalitzi en verd, entra a **Settings > Pages** o a l'enllaç generat a la pàgina principal del repositori.

4. Obriràs la URL pública (tipus https://usuari.github.io/nom-repositori/).
5. Comprova manualment que la pàgina es veu bé, el CSS es carrega, pots afegir/filtrar tasques i el localStorage manté les dades.

El flux final serà:

```
Pull Request
      ↓
CI
      ↓
Tests
      ↓
PASS
      ↓
Merge
      ↓
main
      ↓
Deploy
      ↓
GitHub Pages
```

PAS 13. Gestionar les dependències

El projecte utilitza dependències npm.
Configura Dependabot perquè comprovi periòdicament les actualitzacions de les dependències.
El repositori haurà d'incloure:

```
.github/
└── dependabot.yml
```

La configuració haurà de contemplar les dependències del projecte i les dependències dels GitHub Actions utilitzats, quan sigui aplicable.

PAS 14. Comprovar Dependabot

Comprova que la configuració de Dependabot és visible i correcta.
Si durant el període de treball Dependabot genera una actualització, revisa-la i documenta el procés.
Si no es genera cap actualització durant la pràctica, serà suficient demostrar correctament la configuració.

PAS 15. Actualitzar el README

El README haurà de documentar el projecte i el seu sistema DevOps.
Ha d'incloure, com a mínim:

Descripció

Què fa DevTasks.

Instal·lació

Com instal·lar les dependències.

Tests

Com executar els tests.

GitHub Actions

Quins workflows existeixen i què fa cadascun.

Pull Requests

Quin flux de treball s'ha establert.

Deploy

Enllaç a la web publicada.

Dependències

Com es gestionen les actualitzacions.

Arquitectura

Explicació de la separació entre:

```
app.js
taskManager.js
tests/
```

PAS 16. Comprovar el flux DevOps complet

Abans de l'entrega final hauràs de demostrar el funcionament complet del sistema.
Realitza una petita modificació al projecte.
El procés haurà de ser:

```
1. Crear branca
      ↓
2. Modificar codi
      ↓
3. Commit
      ↓
4. Push
      ↓
5. Pull Request
      ↓
6. GitHub Actions
      ↓
7. Tests
      ↓
8. PASS
      ↓
9. Merge
      ↓
10. Deploy automàtic
      ↓
11. GitHub Pages actualitzat
```

PAS 17. Activitat d'ampliació: CodeQL

Completa el tutorial:
Introduction to CodeQL
i incorpora Code Scanning al projecte.
L'objectiu és que el repositori pugui detectar automàticament determinats patrons de vulnerabilitats en el codi.

Skill aplicat

Introduction to CodeQL
Aquesta activitat és opcional.

ENTREGA 3 — REPOSITORI DEVOPS I EVIDÈNCIES

L'entrega final haurà d'incloure:

1. Repositori GitHub

Enllaç al repositori final.
El repositori haurà de ser accessible per al professorat.

---

2. Aplicació publicada

Enllaç a la versió publicada amb GitHub Pages.

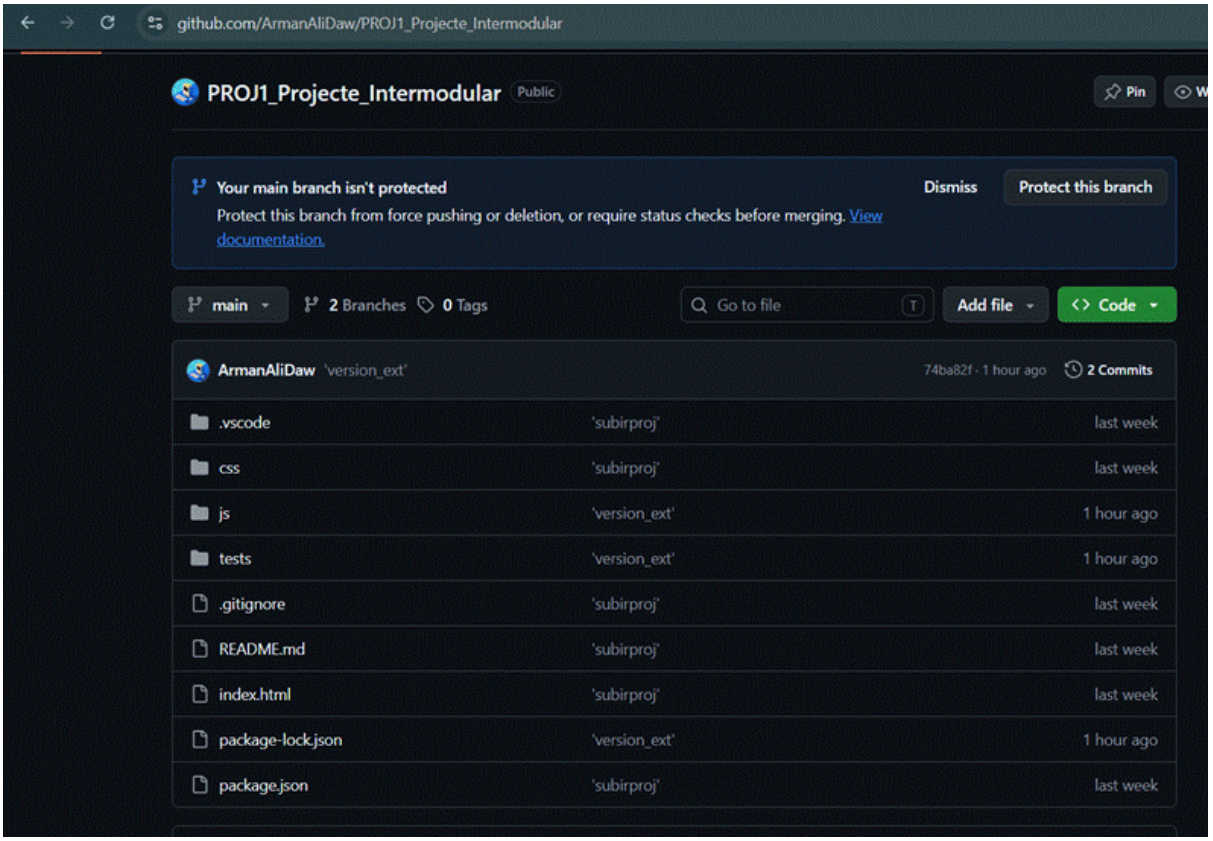
---

3. Evidències

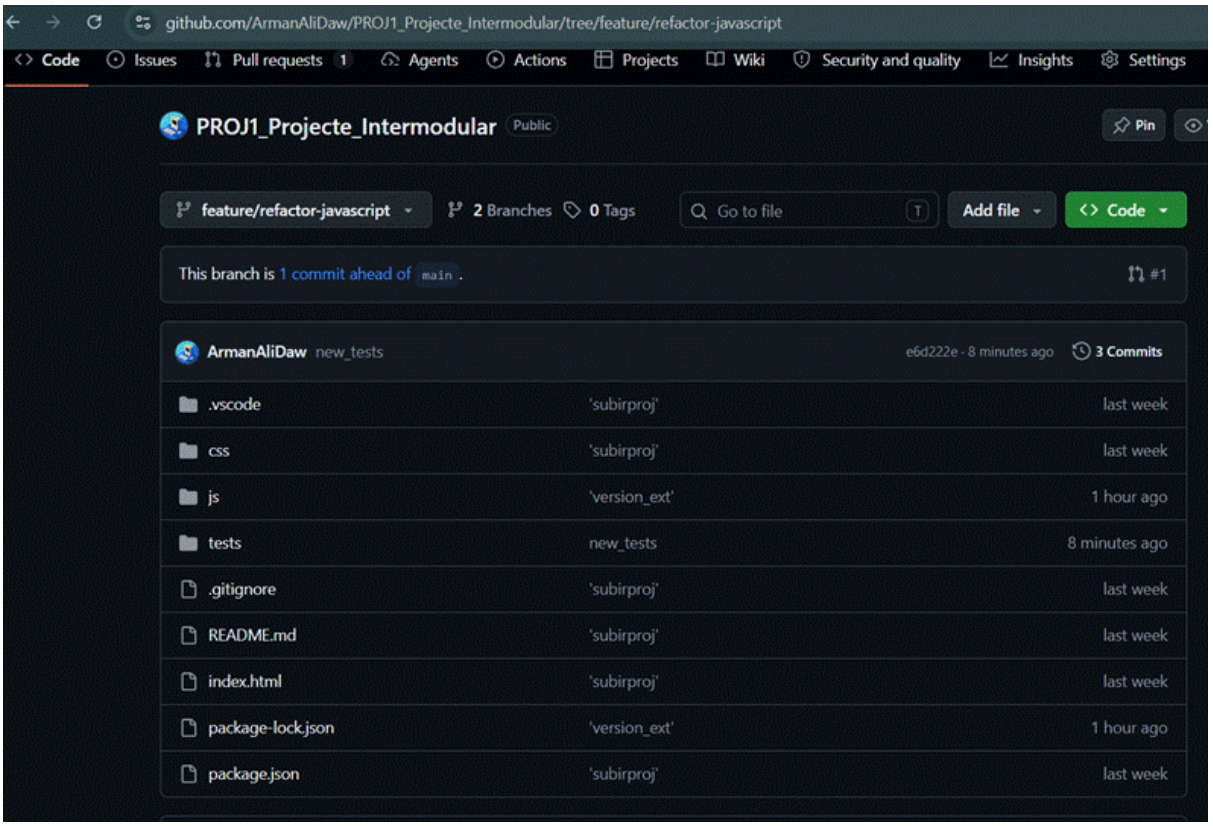
Entrega un document PDF amb captures que demostrin:

Git

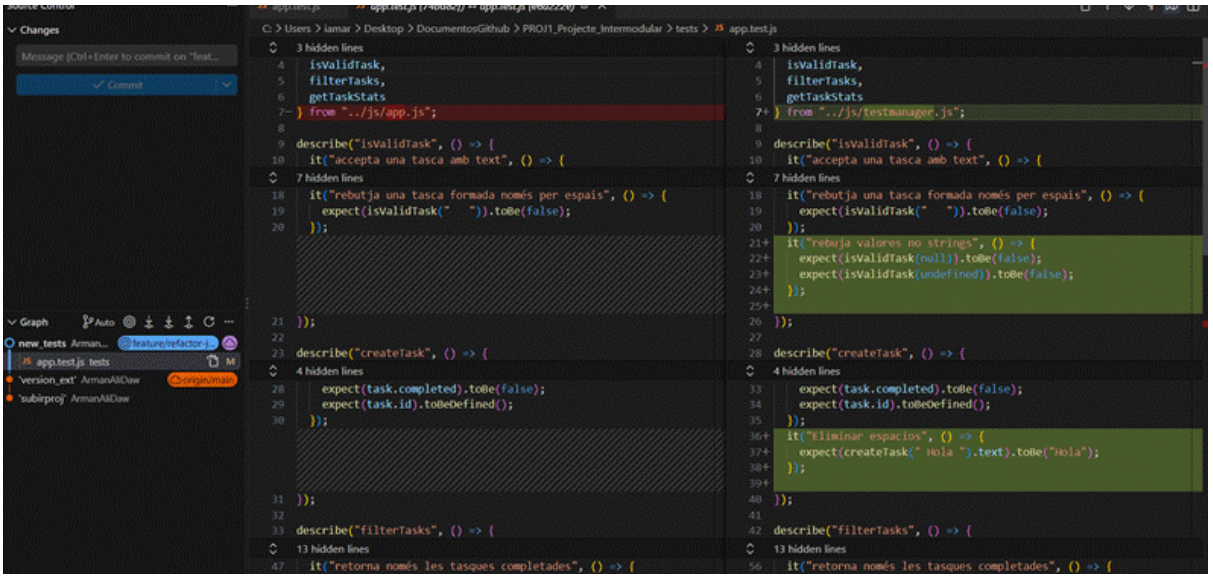
repositori;



branques;

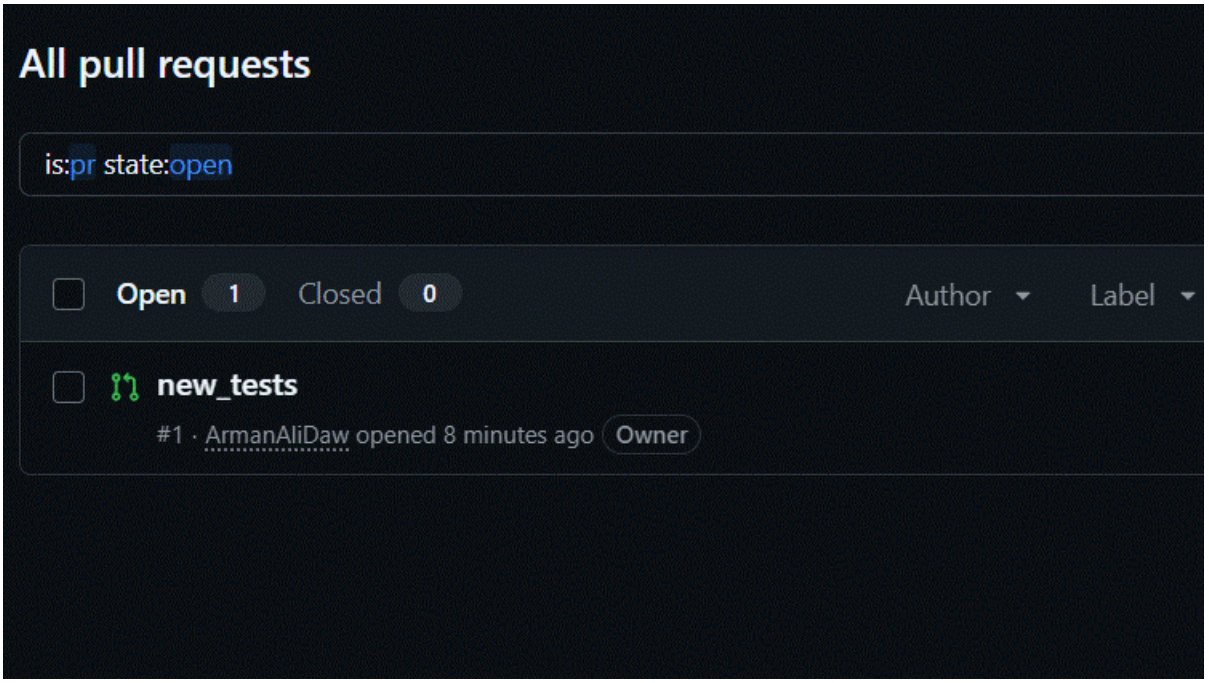


commits.

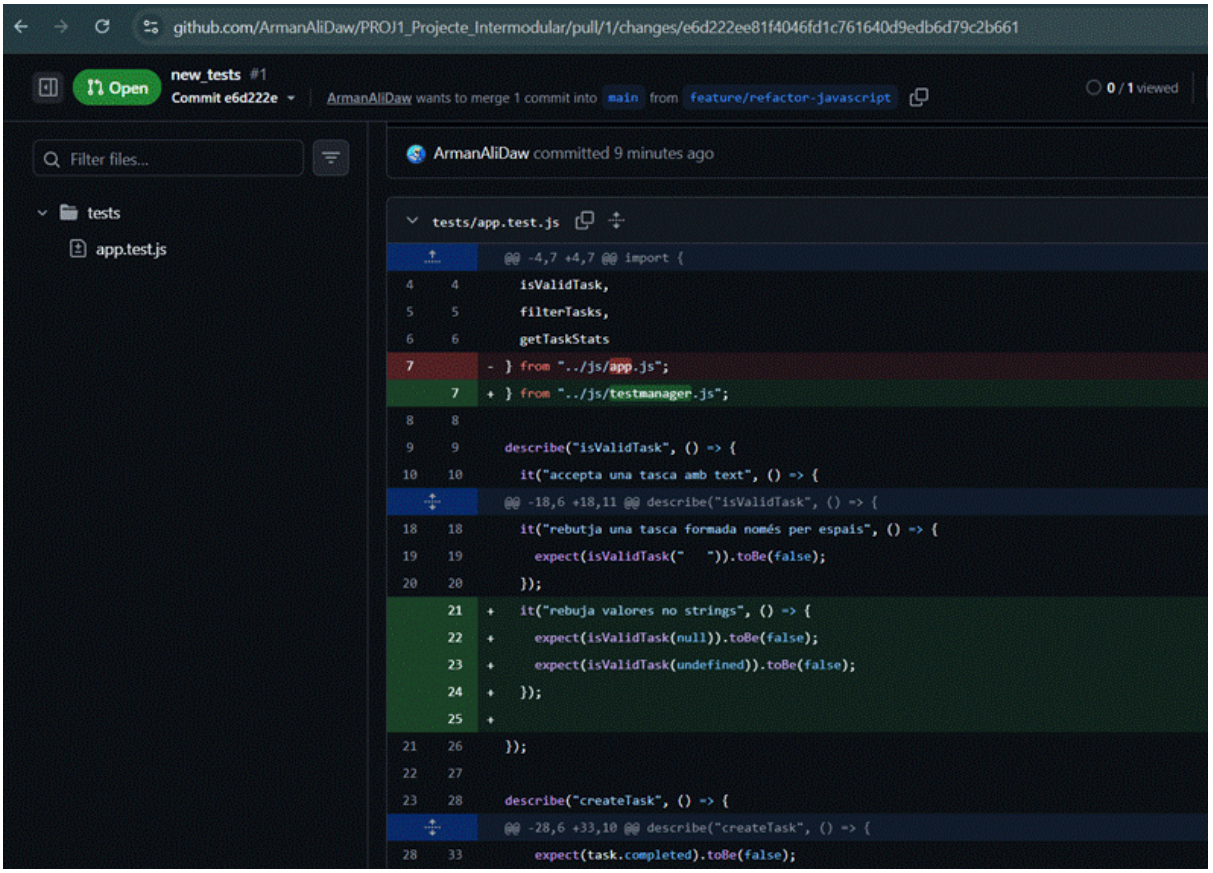


Pull Request

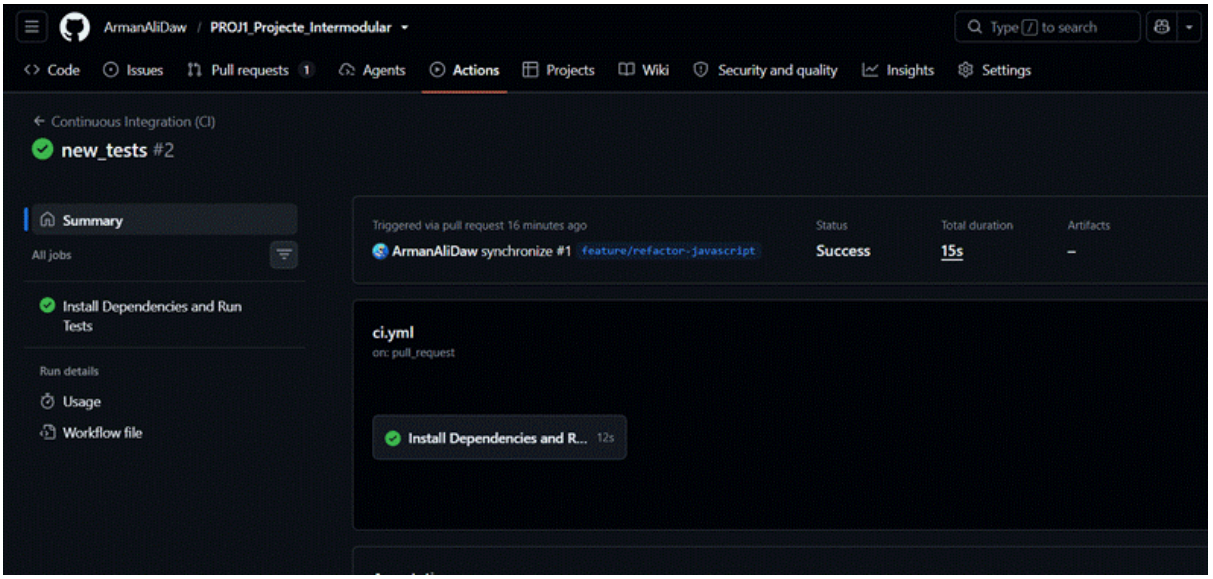
una PR real;



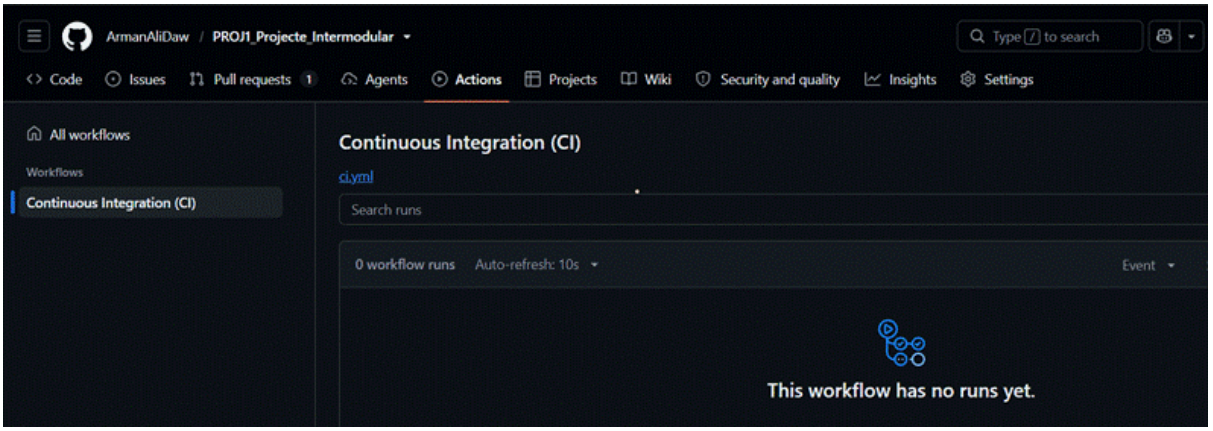
revisió;



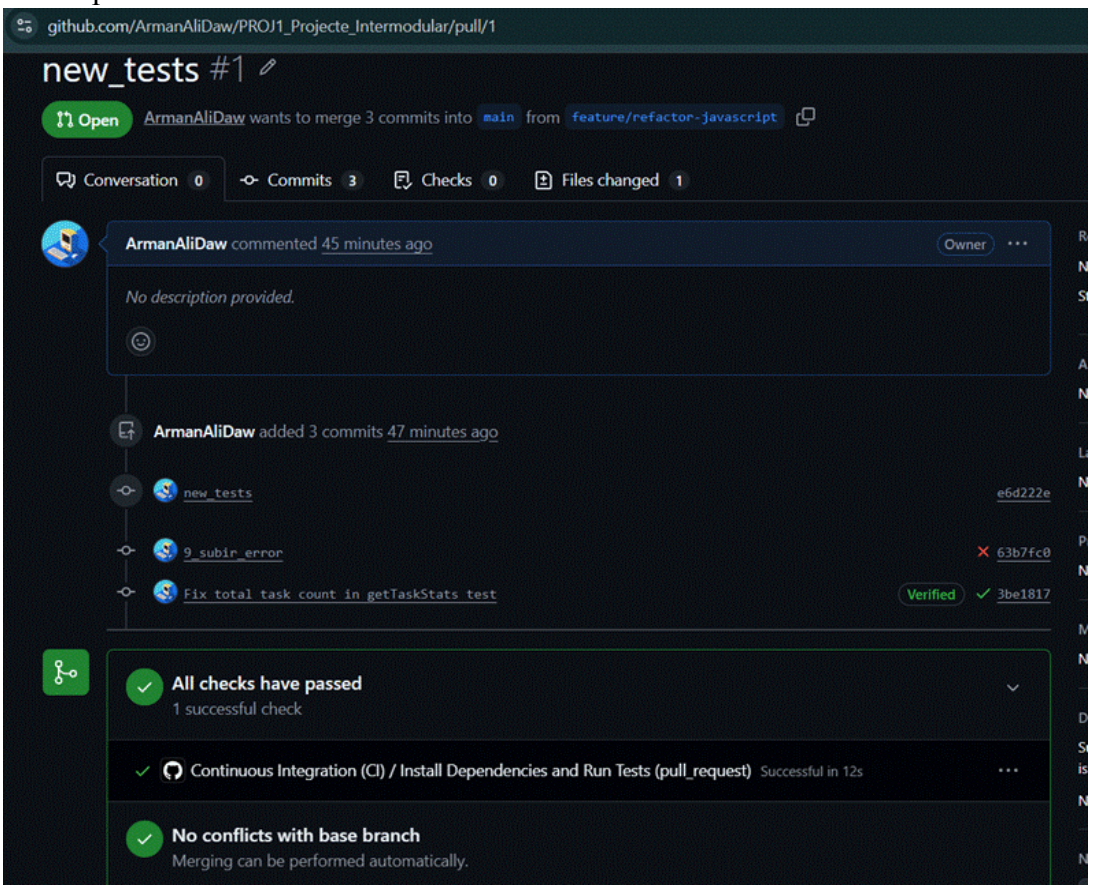
merge.


ArmanAliDaw / PROJ1_Projecte_Intermodular
Type / to search
Code
Issues
Pull requests 1
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Continuous Integration (CI)
new_tests #2
Summary
All jobs
Install Dependencies and Run Tests
Run details
Usage
Workflow file
Triggered via pull request 16 minutes ago
ArmanAliDaw synchronize #1
feature/refactor-javascript
Status
Success
Total duration
15s
Artifacts
–
ci.yml
on: pull_request
Install Dependencies and R...
12s

CI

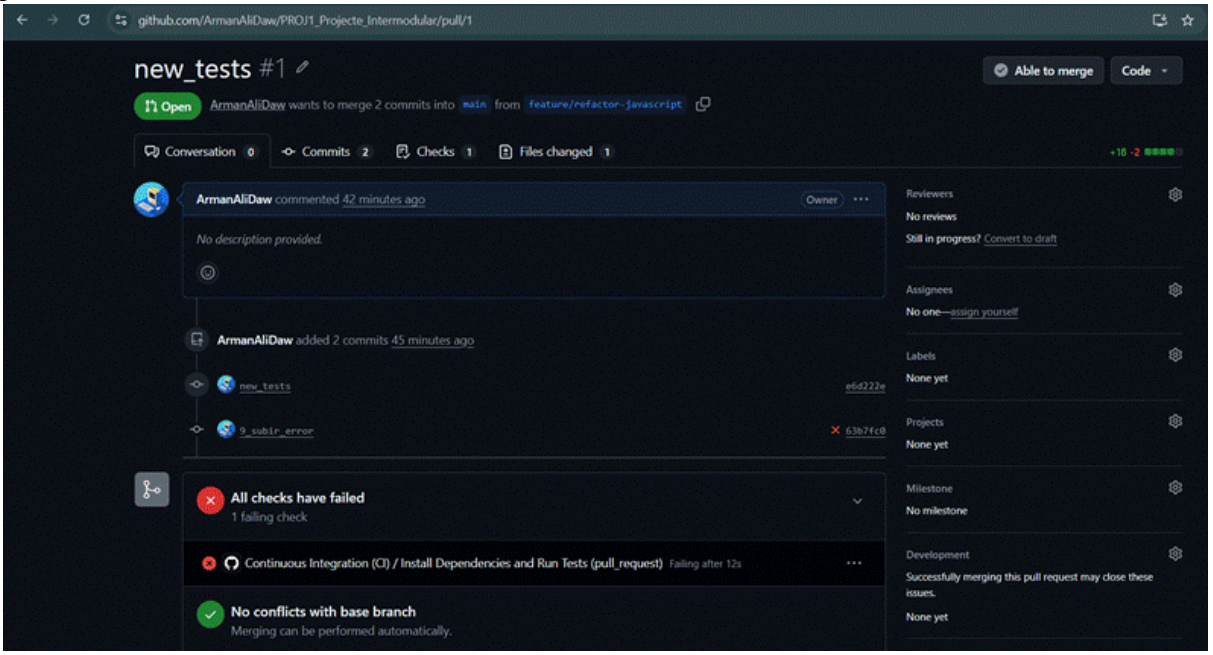

ArmanAliDaw / PROJ1_Projecte_Intermodular
Type / to search
Code
Issues
Pull requests 1
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
All workflows
Workflows
Continuous Integration (CI)
Continuous Integration (CI)
ci.yml
Search runs
0 workflow runs
Auto-refresh: 10s
Event
This workflow has no runs yet.

execució correcta;
tests superats.



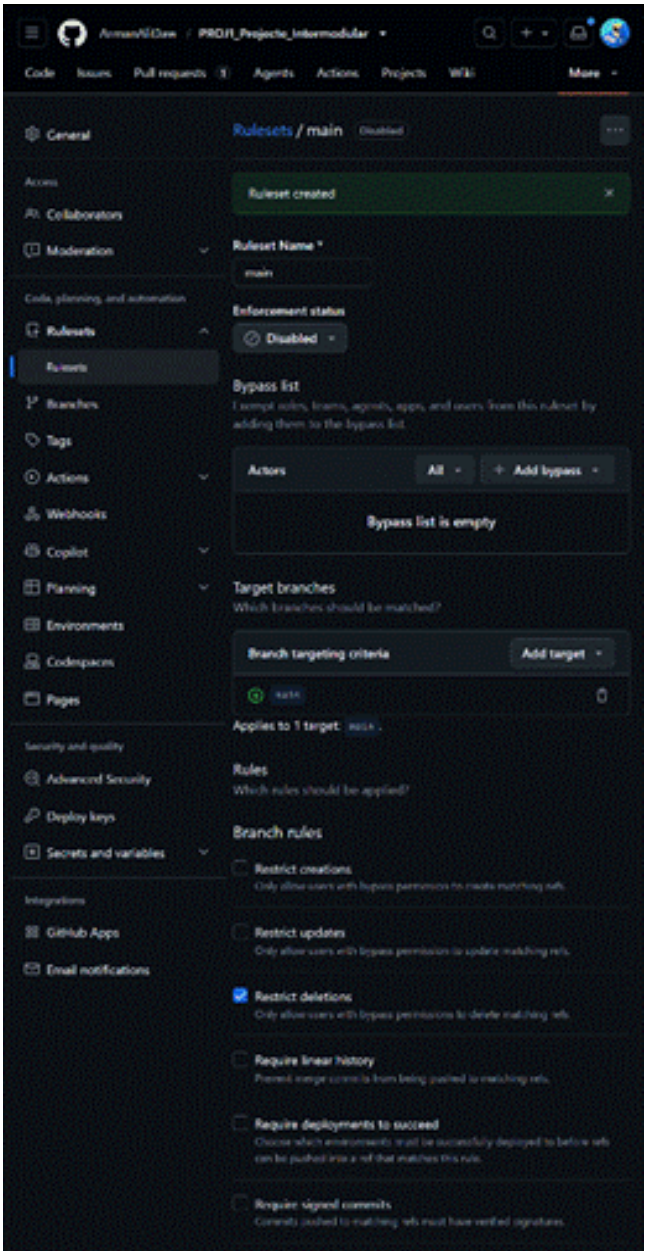
Error

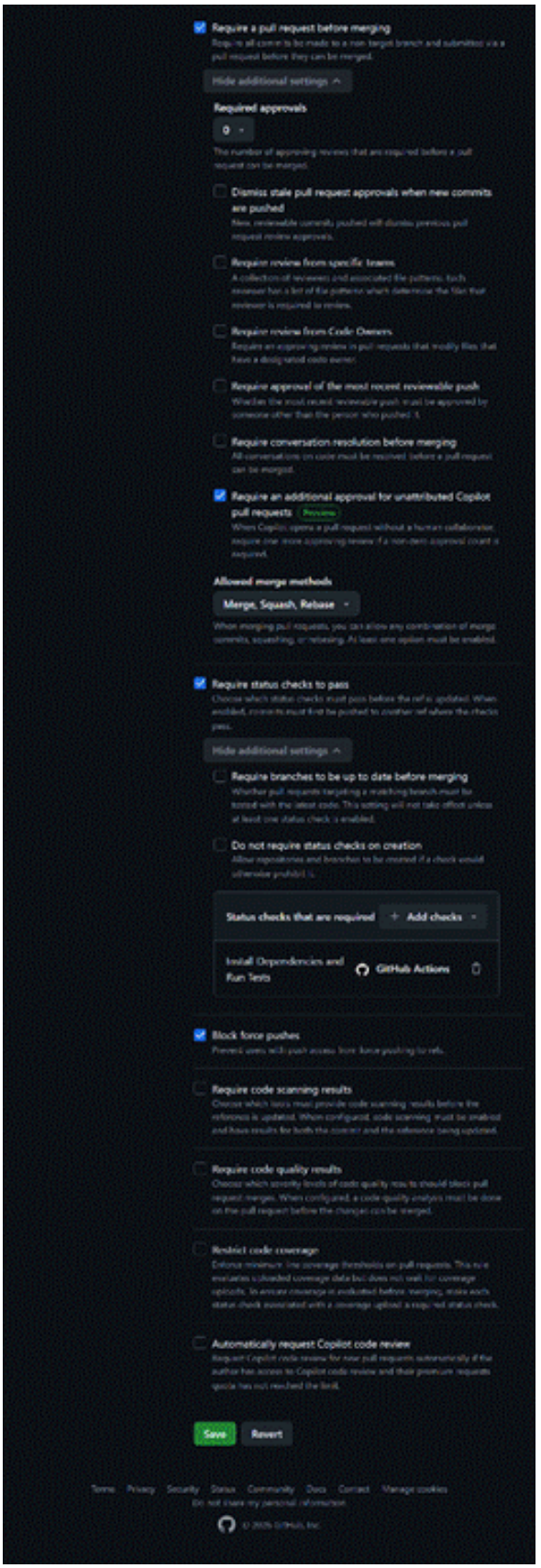
execució amb tests fallits;
posterior correcció.



Branch protection

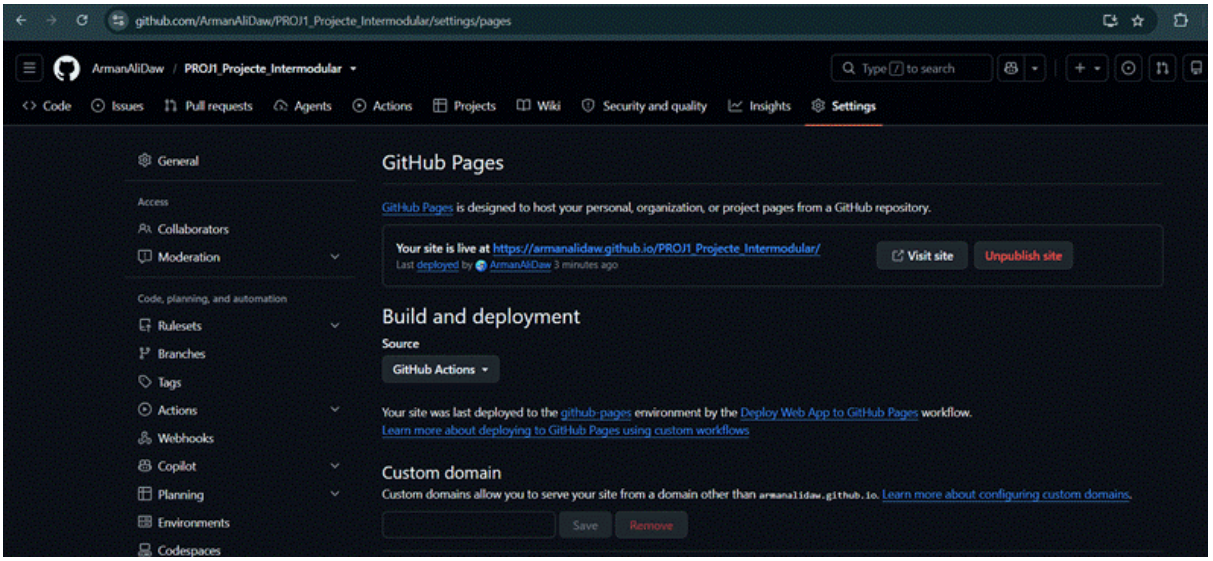
configuració de protecció de `main`.



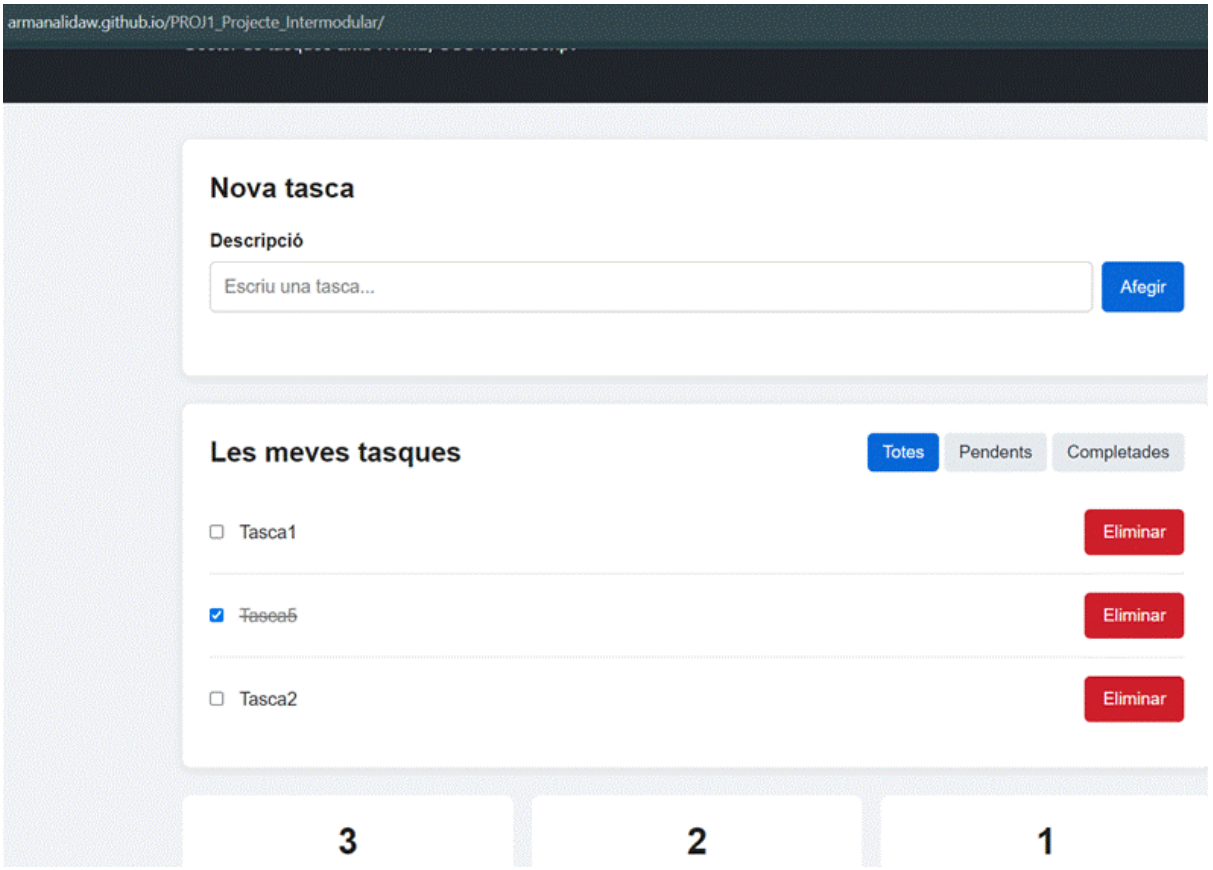


Deploy

execució del workflow;



aplicació publicada.



Dependabot

configuració;

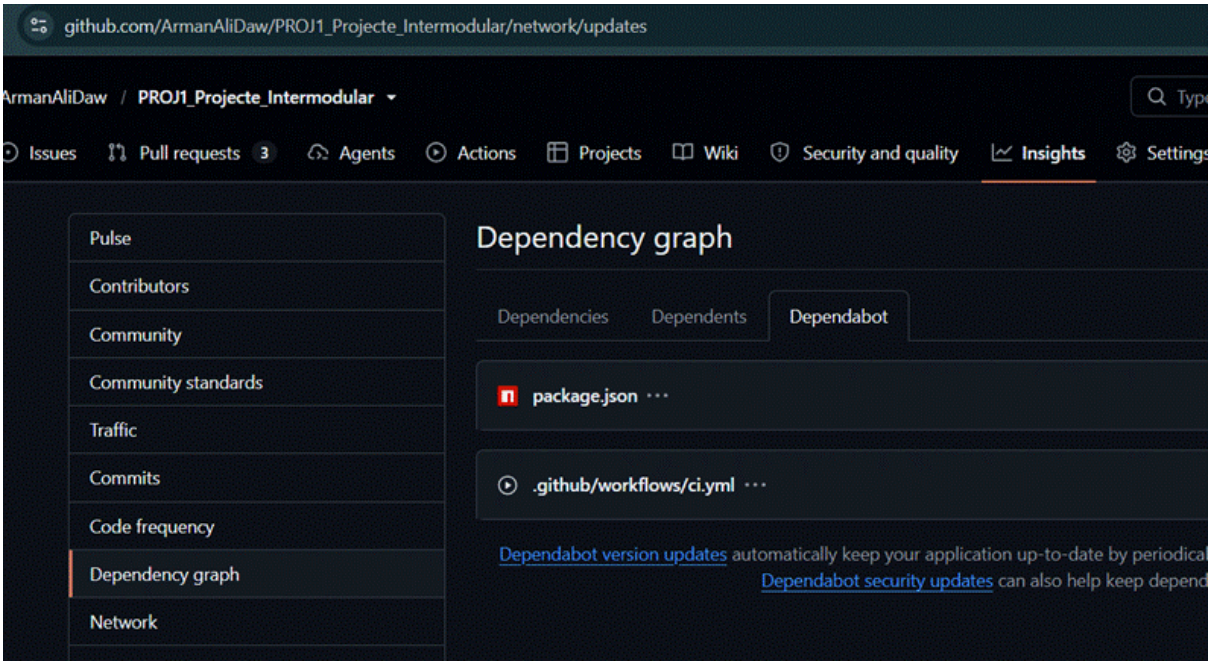
PROJ1_Projecte_Intermodular / .github / dependabot.yml

ArmanAliDaw Create dependabot.yml

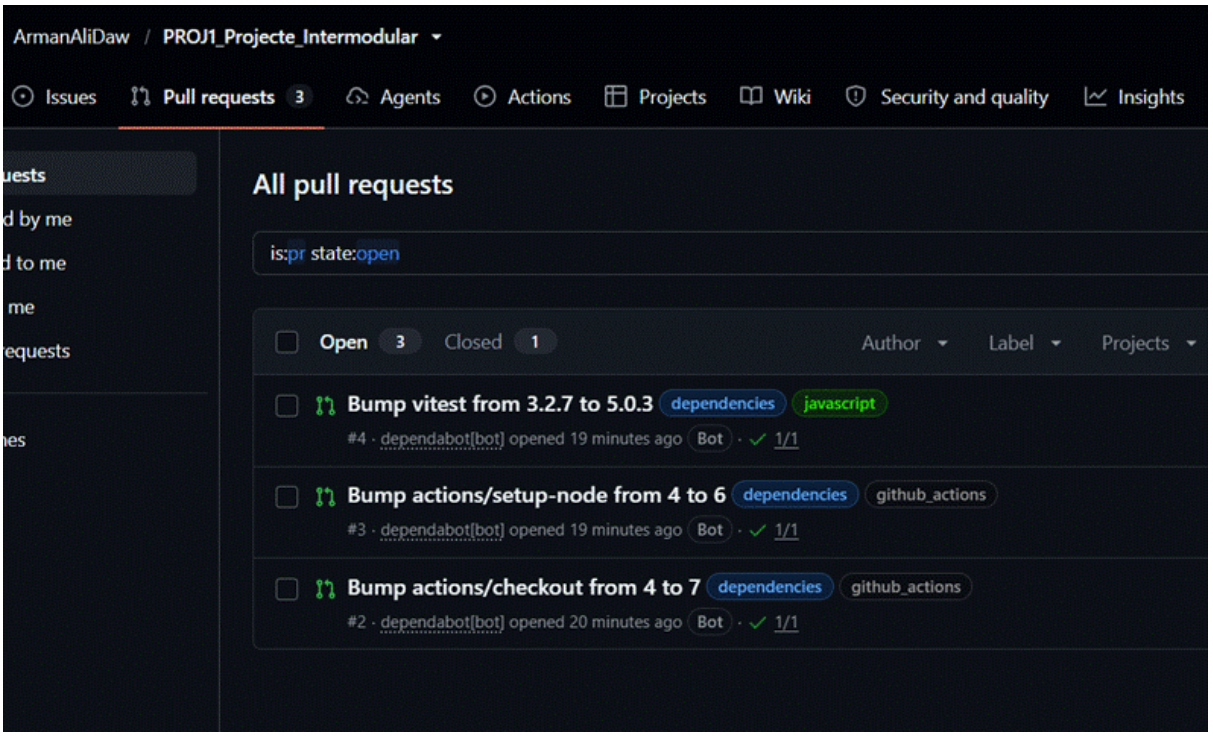
Code Blame 14 lines (13 loc) · 368 Bytes

```yaml
version: 2
updates:
  # Dependències npm del projecte (package.json)
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 5

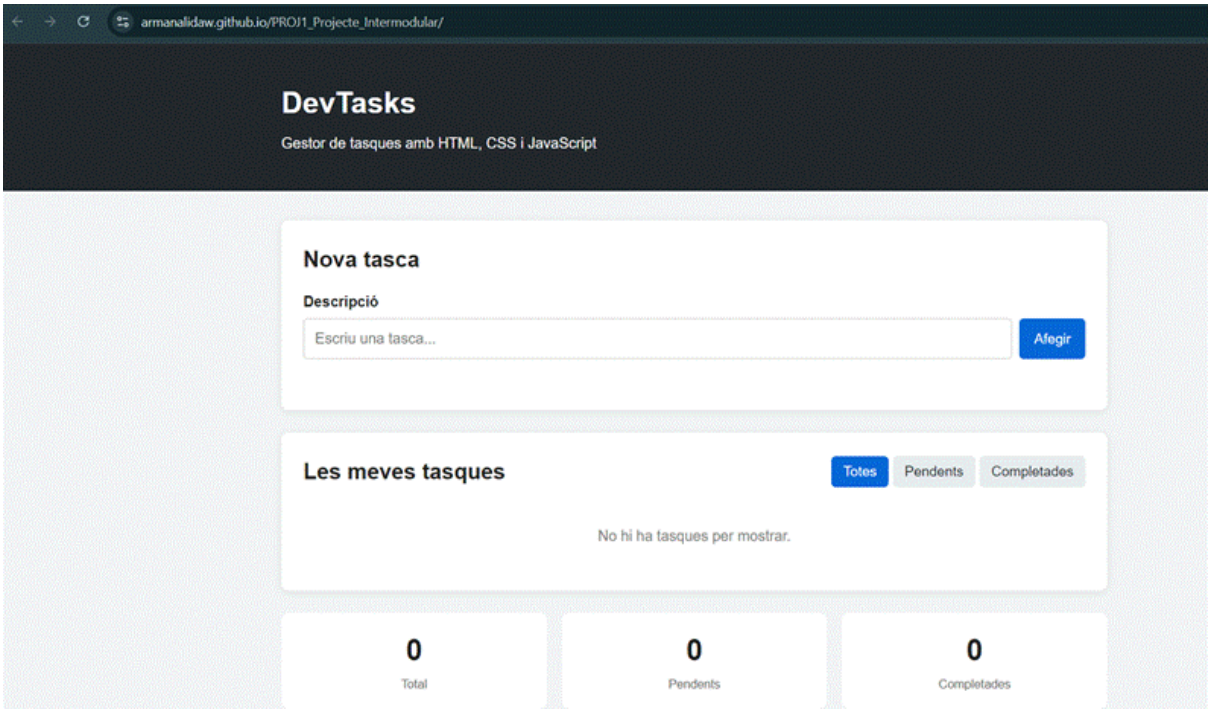
  # Dependències dels GitHub Actions (actions/checkout, actions/setu
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
```

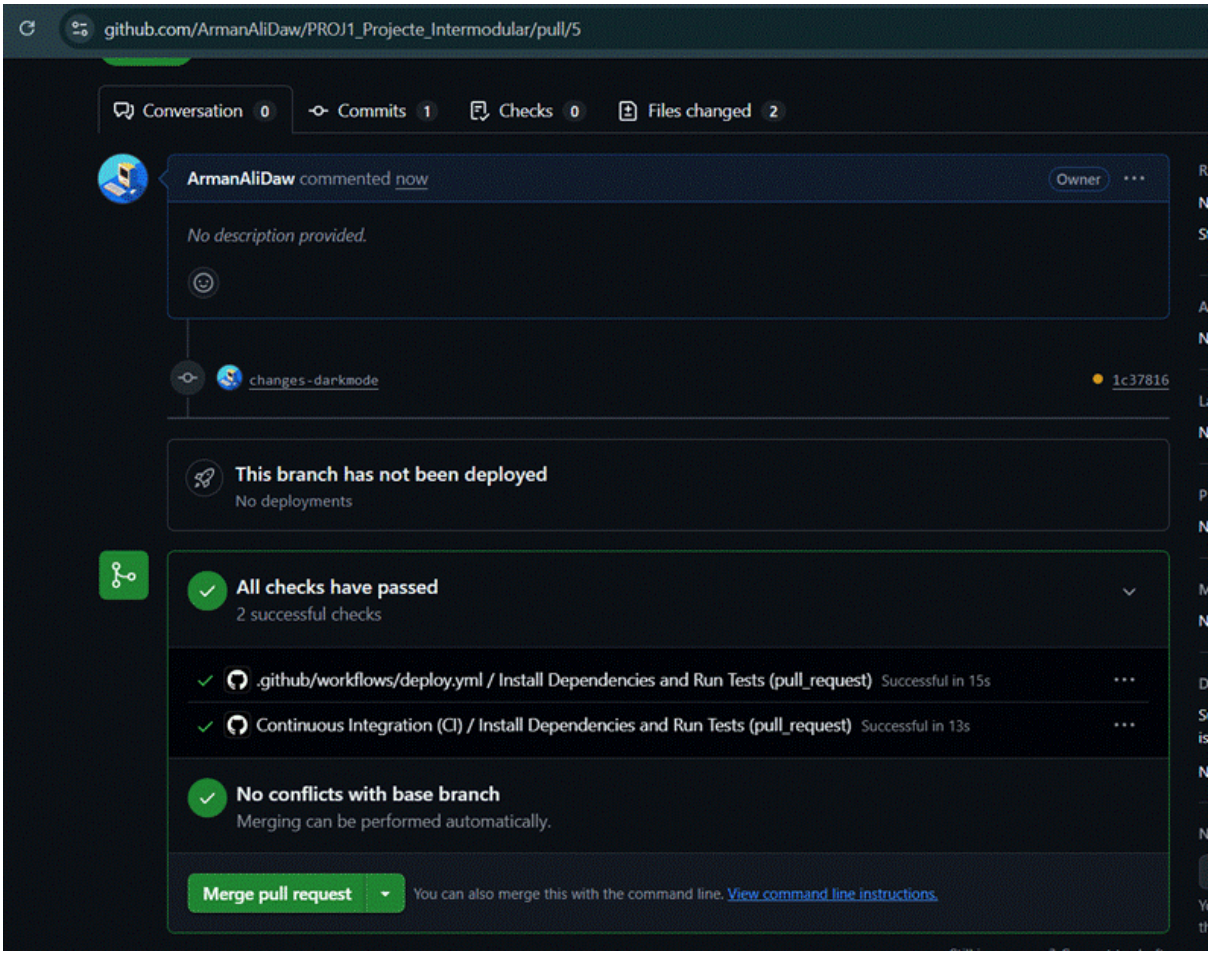


actualització/PR si se n'ha generat alguna.

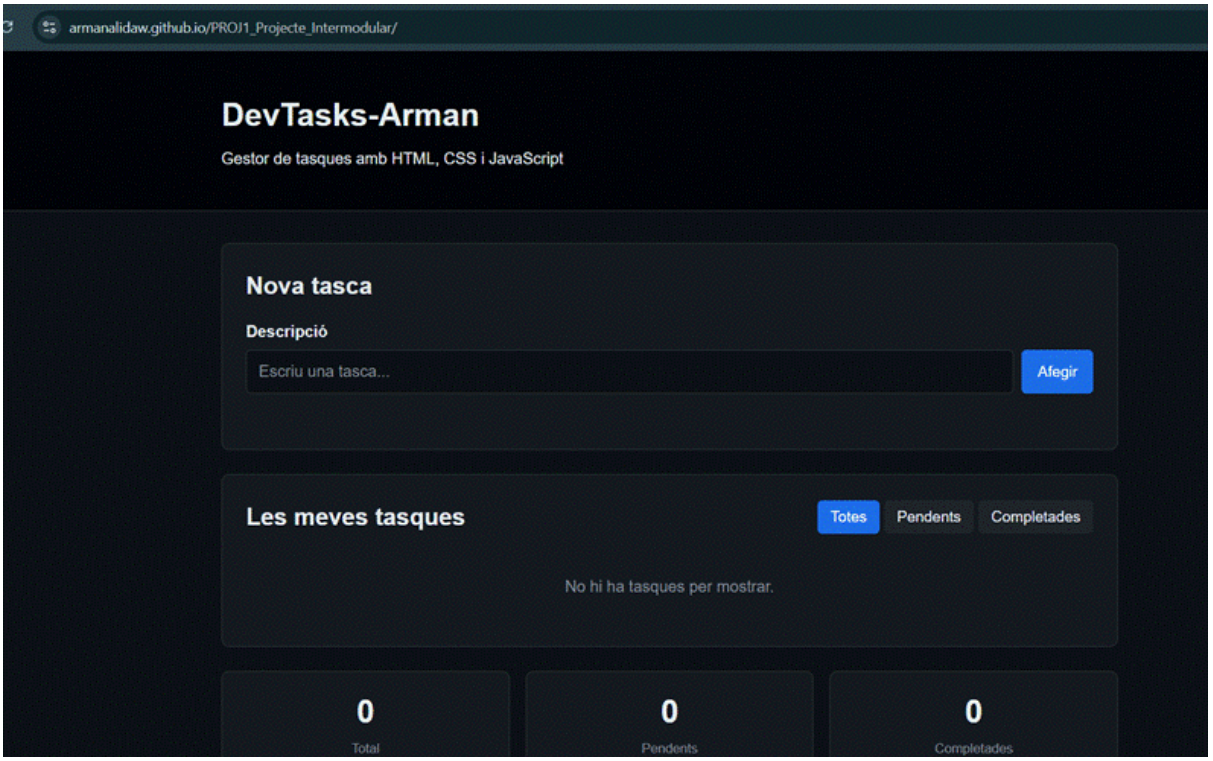


Aplicació



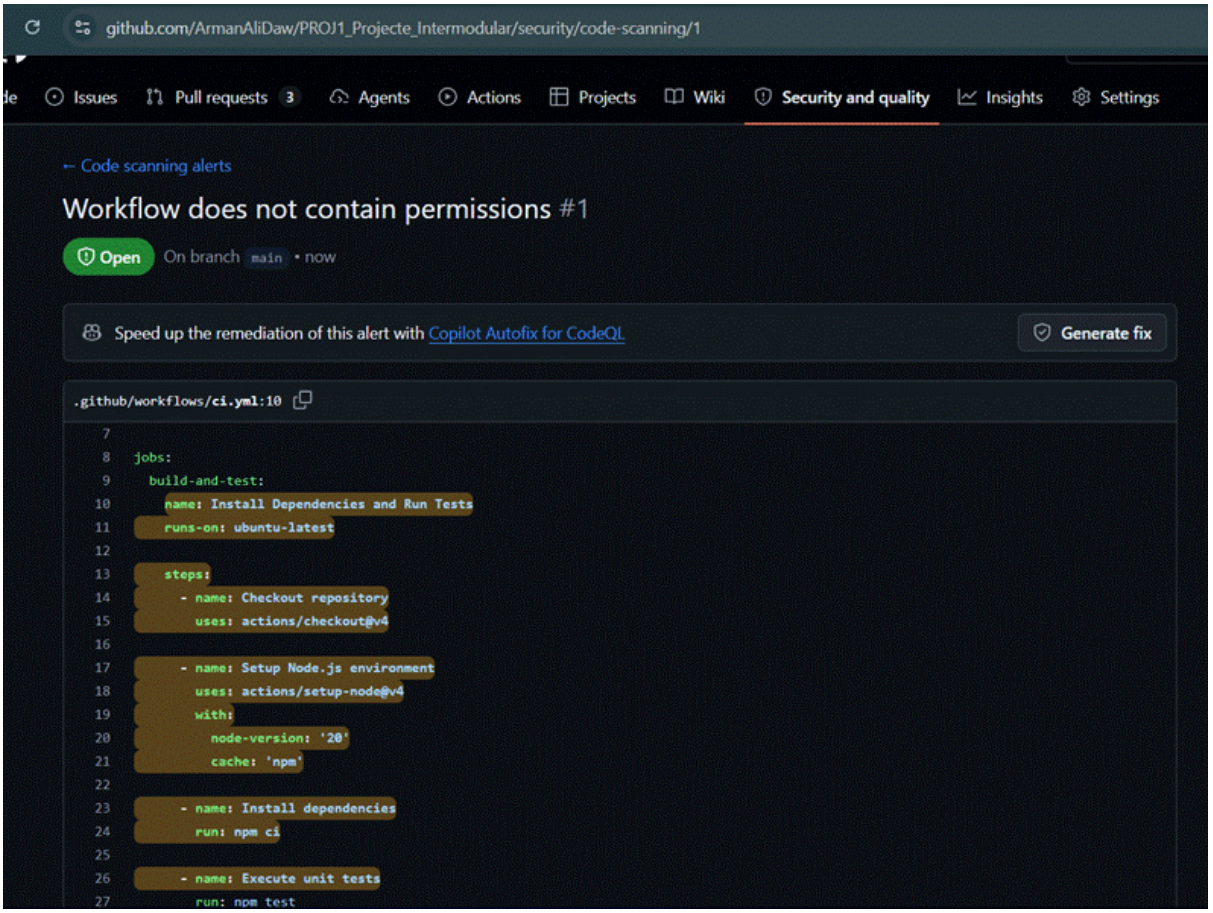


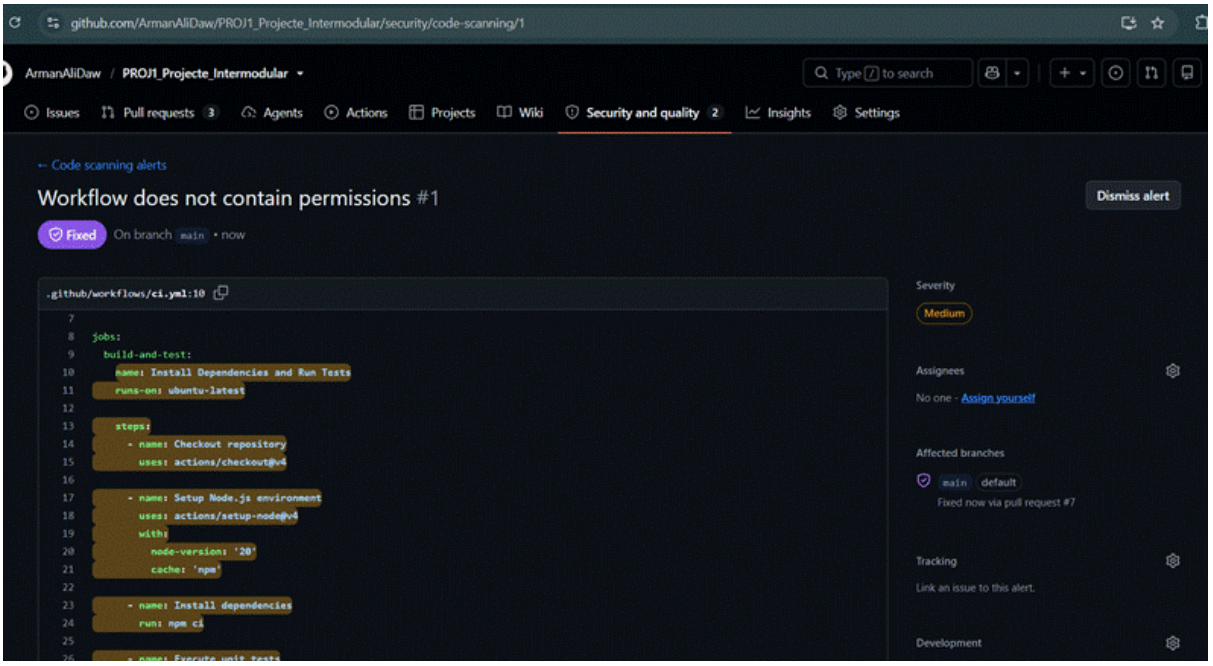
web publicada i funcionant.



Opcional

CodeQL / Code Scanning.



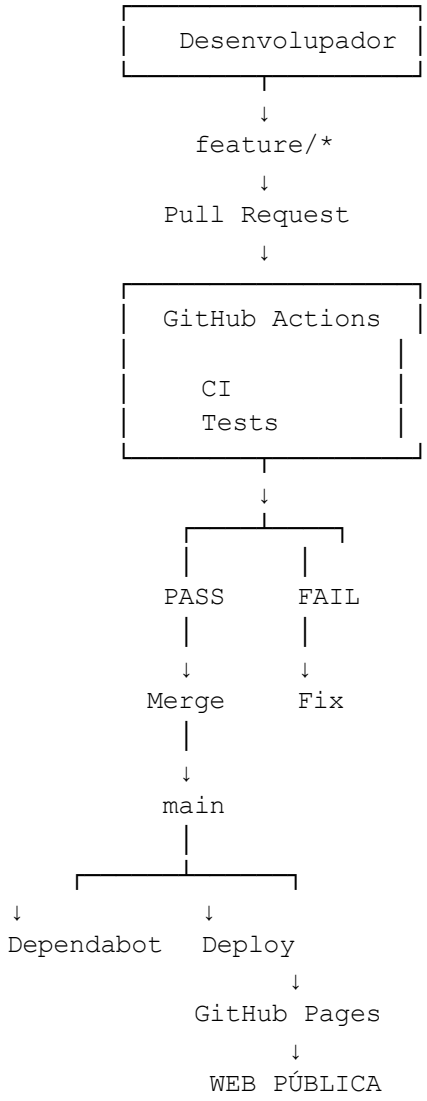


RESULTAT FINAL

Al final de la pràctica hauràs transformat un projecte web convencional:

```
HTML
CSS
JavaScript
```

en un projecte amb un flux DevOps:



El producte final no és només l'aplicació DevTasks.

El producte final és el projecte web + el repositori + el flux de treball automatitzat + les evidències que demostren que funciona.